claude-session-publisher — User Manual

transcript_archiver.py v2.6.2

Contents

claude-session-publisher — User Manual

`transcript_archiver.py` turns a Claude conversation into a self-contained document — HTML, plain text, Markdown, LaTeX or PDF — with a fidelity report that reconciles every source record against what the page shows. This manual is the complete reference: every option, every output, every feature and every known limitation. The README is the product page; `AGENTS.md` is the same information written for an AI agent driving the tool.

Single file, Python 3.9+, standard library only. No install step:

```
python transcript_archiver.py --version
python transcript_archiver.py --help
```

1. Quick start

```
# archive one Claude Code session to HTML (the default format)
python transcript_archiver.py <session-id>
```

```
# every format at once
python transcript_archiver.py <session-id> --format html,text,markdown,latex,pdf

# rebuild the index page of everything on disk
python transcript_archiver.py --index

# try it on the shipped showcase conversation
python transcript_archiver.py 0000c0de-cafe-4000-8000-00000000f00d \
    --projects-root examples --archive-dir demo --format html,markdown,pdf
```

The session id is the `.jsonl` file name under `~/.claude/projects/<project>/`. `--index` lists every session it can find with its id and title, so run it first if you do not know the id.

2. Sources

Source	How	Notes
Claude Code CLI / desktop app	default; sessions under `--projects-root` (`~/.claude/projects`)	native format
Claude Code web/mobile bridged to your machine	same	bridge records are chain-resolved into one conversation
Claude Desktop cowork (local agent mode)	`--cowork-root` (auto-detected per platform)	same record schema, different base directory; `audit.jsonl` is skipped. Tested against synthetic data only
claude.ai chats, Claude Desktop chat, mobile app	`--import-claude-ai conversations.json`	from Settings → Privacy → Export data. Standalone chats only; no Project conversations, no usage or model data (the page says so)
Claude Code cloud sessions never bridged	not archivable	nothing is written to your disk

Cowork auto-detection: `%APPDATA%\Claude\local-agent-mode-sessions` on Windows, `~/Library/Application Support/Claude/local-agent-mode-sessions` on macOS, `~/.config/Claude/local-agent-mode-sessions` elsewhere. Pass `--cowork-root ""` to disable.

3. Command-line reference

Every input and output is reachable from the command line; nothing is hard-coded. `--help` prints each option with its default.

Positional

`session_id`	transcript UUID (the `.jsonl` filename). Optional with `--index` or `--import-claude-ai`.

Discovery and placement

Option	Default	Meaning
`--projects-root DIR`	`~/.claude/projects`	where Claude Code writes sessions
`--cowork-root DIR`	auto per platform	Claude Desktop cowork sessions, merged into discovery when the directory exists; `""` disables

Option	Default	Meaning
`--archive-dir DIR`	`$CLAUDE_ARCHIVE_DIR` or `~/claude-archives`	where archives, `index.html` and `logs/` go
`--out PATH`	—	output path **stem** for a single archive; each format adds its own extension (`--out report.pdf --format html` writes `report.html`). Overrides `--archive-dir` naming
`--title TEXT`	the session's own `ai-title`	page title; also drives the file name slug. Re-archiving with a different title writes a new file
`--summary-file FILE`	placeholder	HTML fragment (`h3`/`ul` blocks) rendered as the hand-written session summary

Content

Option	Default	Meaning
`--format LIST`	`html`	comma-separated: `html`, `text`, `markdown` (or `md`), `latex`, `pdf`
`--tool-output on|off`	`on`	include tool input and output. Independent of `--format`. `off` reduces each tool call to one labelled line — usually what you want for LaTeX/PDF
`--max-tool-output N`	`16384`	elide the middle of any tool output longer than N characters; every elision is counted on the page. `0` = never
`--full`	off	never elide (same as `--max-tool-output 0`)
`--subagents on|off`	`on`	render subagent transcripts as appendix sections. With `off` they are still listed in the fidelity report and their usage still counts
`--no-follow-chain`	off	archive exactly the id given even if a more complete continuation exists
`--fragment`	off	with `--format latex`: body only, no preamble, transliterated to compile under pdflatex as well as XeLaTeX. Cannot be combined with `pdf`
`--paginate N`	`0`	split the HTML into pages of N turns; page 1 keeps the summary, usage and fidelity sections; the sidebar links across pages

Index

Option	Meaning
`--index`	rebuild `index.html` in `--archive-dir` and exit
`--watch SECONDS`	with `--index`: regenerate every SECONDS (minimum 30) until Ctrl+C, and stamp the page to reload itself

claude.ai import

Option	Meaning
`--import-claude-ai FILE`	import conversations from a claude.ai `conversations.json`
`--conversation TEXT`	only conversations whose name or uuid contains TEXT (case-insensitive)
`--list-conversations`	list the export's conversations and exit

Output control and logging

Option	Meaning
`--verbose`	per-step detail (files parsed, compile passes, audit log path)
`--quiet`	print nothing but warnings; the audit log still records everything
`--log-dir DIR`	where the per-run audit log goes (default `<archive-dir>/logs/`)
`--version`	print the archiver version and exit
`--help`	option reference

Invalid combinations are rejected before anything is written: `--watch` without `--index`; `--conversation`/`--list-conversatio` without `--import-claude-ai`; `--fragment` without `latex` or together with `pdf`; `--verbose` with `--quiet`; an unknown `--format` value.

4. What is produced

Files

In `--archive-dir` (or at the `--out` stem), one file per format: `<session-id>_<title-slug>.html|.txt|.md|.tex|.pdf`. A `--fragment` LaTeX body is `<stem>_fragment.tex`. Paginated HTML adds `<stem>_p2.html`, `<stem>_p3.html`, …. claude.ai imports are named `<uuid-prefix>_<slug>`. `--index` writes `index.html`. Every run writes `logs/<timestamp>_<label>.log`.

When a session is a resumed or bridged conversation, the file is named after the transcript actually archived (the most complete file in the chain), and the page records which id was requested.

The page

Every format carries, in this order: the **session summary** (hand-written via `--summary-file`, or a placeholder), **usage and cost**, the **fidelity report**, then the **transcript**, then **subagent transcripts** as appendices.

Turn kinds and how each format shows them:

Turn	HTML	text / Markdown	LaTeX / PDF
Human prompt (Pn)	right-aligned bubble, verbatim, monospace when columnar, URLs linked	verbatim, never re-wrapped (Markdown: fenced)	verbatim box
Claude response (Rn)	markdown rendered	prose re-wrapped (Markdown: live markdown)	markdown → LaTeX
Thinking	collapsed; empty in practice (see §7)	labelled	labelled box
Tool call	collapsed input/output, error and pending states, screenshots	full I/O or one line (`--tool-output`)	full I/O or bare title box
Pasted image	embedded	announced as omitted	announced as omitted
Harness / system / event	collapsed lane with the classification evidence	labelled blocks	labelled boxes
Subagent transcript	collapsible appendix, linked from the spawning call	appendix section	appendix section

Reference tags

Every human prompt is `P1, P2, …` and every response `R1, R2, …`, sequential within the document; subagent turns are prefixed `A1.`, `A2.` (so `A2.R4`). In HTML the tags are anchors: `page.html#P32` deep-links to the prompt.

Fidelity report

Every source record is **rendered** (produced one or more turns), **folded** (a tool result absorbed into its call), or **counted** (metadata with no transcript content — and corrupt lines). The three numbers are reconciled against the source record count on the page; if they do not add up the page says so instead of hiding it. The report also lists records by type, content blocks, what was rendered and what was counted, the human-vs-injected evidence per record, the subagent files, and caveats (empty thinking blocks, unresolved tool calls, snapshot time versus the source's last record).

Usage and cost

Tokens per model deduped per `requestId` — one API response is written as several records repeating the same usage, and summing them over-reports output ~2.3× on tool-heavy sessions. Cache reads, 5-minute and 1-hour cache writes are separated, and a cost is estimated at **public list rates** from the `PRICING` table at the top of the script (cache reads at 0.1× input, writes at 1.25× / 2×). It is not what a subscription bills. Models the table does not know are reported as "no list price". Subagent usage is merged in.

Reported cost. Claude Code 2.1.9x also writes its own meter into the session file (`cost-state` records: running cost, per-model cost, lines added and removed by tools). When present, the page shows that figure as a *reported cost* column beside the list estimate, a session-info row, and `reported_cost_usd`, `reported_cost_runs`, `reported_cost_partial`, `lines_added`, `lines_removed` in the embedded metadata; the text, Markdown and LaTeX formats carry the same sentence. The meter is **per process**: every `claude --resume` starts a new counter, and runs made before the record existed wrote none — so the figure is the sum of the last snapshot of each run (gathered from every file of a resumed session's chain) and is flagged **partial** when the session began more than a minute before its first metered run. In that case the page says which spend is not covered and the index keeps showing the list-price estimate; otherwise the index shows "$X reported". A run that Claude Code could not price fully is noted ("the reported total is a floor"). In practice the meter has come out ~30 % below the list-price estimate on a one-run session.

The HTML page's controls

Sidebar: **search** (hides turns whose text does not match), **filter** (narrows the contents list; keyboard `/`), lane toggles (thinking, tools, harness, events, subagents), expand/collapse all, **theme toggle** (light or dark, remembered per browser; follows the OS until you choose), session facts, contents. Keys: `j`/`k` jump between human turns.

The index

`--index` scans every session on disk and marks each **archived**, **stale** (source has newer records than the archive), **covered** (resumed into another transcript that is archived), **legacy v1**, or **not archived**; lists archives whose source is not on disk (claude.ai imports, deleted transcripts); and shows an activity column whose ages decay in the browser. Headers sort on click. `--watch` keeps it regenerating. A first `--index` into a directory that does not exist yet creates it.

Search across every archive. The index page carries a search box over every human prompt of every archive — all pages of a paginated archive and subagent prompts (`A1.P1`) included — read back from the archives' own HTML at index time, so archives written by earlier versions and claude.ai imports are covered alike. Typing two or more characters lists the matching prompts (session, tag, title, highlighted snippet; first 200 shown), each linking straight to the prompt's anchor on its page, and narrows the session table to the sessions that matched. Prompts are capped at 400 characters in the index; Claude's responses are searchable within each page, not across archives (see limitations).

5. LaTeX and PDF

Requirements: a TeX installation providing `xelatex`, `fvextra`, `tcolorbox`, `booktabs`, `enumitem`, `xcolor`, `hyperref` and the DejaVu fonts (TeX Live `scheme-full` has all of them). Fonts are loaded **by file name from TeX Live**, not from the system, so output does not depend on the machine's font database.

- `pdf` = the standalone LaTeX compiled by `xelatex` twice (for the table of contents); `.aux/.log/.out/.toc` are removed on success and the `.tex` is kept only if `latex` was also requested. On failure the last 30 lines of the log are printed and the `.tex` stays for inspection.
- `--fragment` emits a body for `\input` into your own document. It is engine-neutral: Greek becomes math (Γ → `$\Gamma$`), sub/superscripts become math, arrows and box drawing become ASCII, accents are stripped to the base letter. Your preamble needs `\usepackage{fvextra} \usepackage{xcolor} \usepackage{enumitem} \usepackage{booktabs} \usepackage[most]{tcolorbox}`. The turn environments are defined with `\@ifundefined`, so you can restyle them from your preamble.
- Emoji and other glyphs no TeX font can set, and C0/C1 control bytes (NULs from UTF-16 console captures, backspaces), are removed and **counted in the document**. Lines over 500 characters are hard-wrapped so TeX can typeset them; the count is stated.
- A turn longer than 1,500 typeset lines (a huge paste or tool output) is split into consecutive boxes titled *(part k/n)*: one breakable box holding it whole exhausts TeX's memory. The document states how many turns were split; nothing is omitted.
- Validated by a full pass over a real 62-session archive (5,109 pages, `--tool-output off`, August 2026).
- Cost of full tool I/O, measured: a 636-record session → 643 pages in about four minutes; a 1,655-record session is 92 pages with `--tool-output off` and 260 with it on.

6. Logging and audit

Console: progress lines by default; `--quiet` silences them; `--verbose` adds per-step detail. Warnings always go to stderr. Every invocation writes `<archive-dir>/logs/<YYYYMMDD-HHMMSS>_<label>.log` (or under `--log-dir`) containing the archiver and Python versions, the exact command line, the working directory, start and end times, every console message, and the outcome (`ok`, `failed: …`, `crashed: …`, `interrupted`). Logging never aborts a run.

7. Known limitations

These are the honest edges. Each is stated on the page where it applies.

- **Thinking text is never in the transcript.** Claude Code requests thinking with `display: "omitted"`; every thinking block on disk is empty. The archive shows *that* Claude thought at a point, never what it thought.
- **The list cost is an estimate**, not a bill; the `PRICING` table is hardcoded (August 2026 rates) and must be edited when rates change. The **reported cost** is Claude Code's own figure but is per process: sessions resumed across runs, or begun before Claude Code 2.1.9x, are covered only in part and say so (`partial`).
- **A live session is off by one tool call**: archiving from inside the session leaves the archiver's own call unresolved; the page says so.
- **The claude.ai export contains standalone chats only** — no Project conversations, no cowork sessions, no usage or model names. It targets the export schema as of mid-2026.
- **Cowork discovery follows the documented layout** but was tested against synthetic data only; the local cowork store does not survive an app reinstall.
- **Cloud sessions never bridged to your machine cannot be archived.**
- **Text and Markdown cannot carry images**; they are announced as omitted. LaTeX/PDF likewise; the HTML holds them.
- **Markdown rendering covers Claude's own prose** (headings, lists incl. nested, tables, code fences of any length, quotes, inline code/bold/ italic/strike/links), not arbitrary CommonMark: no HTML passthrough, no reference links, no footnotes; table cells split on every `|`.
- **Human-vs-injected classification** is authoritative on records carrying `promptSource` / `origin.kind`; older records fall back to text markers, and the evidence used is listed per record in the fidelity report.
- **Timestamps are local** to the archiving machine (hover for UTC in HTML).
- **Re-archiving with a different `--title` writes a new file** beside the old one rather than overwriting it.
- **Scale**: every run re-reads all `.jsonl` files under the roots to resolve chains; the index compares uuid sets pairwise. Fine for hundreds of sessions; slow for thousands.
- **Platforms**: developed and validated on Windows; the suite runs on Linux in CI; macOS untested beyond the cowork path being defined.
- **Live index decay is one-directional**: a session can go quiet on screen but cannot become active without regeneration (`--watch`).
- **Cross-archive search covers prompts, not responses** (and the first 400 characters of each prompt). Responses are searchable within a page. Indexing responses would multiply the index file's size and is deferred.

8. Tests

```
python tests/test_archiver.py
```

260 checks against the synthetic sessions in `examples/` (no real transcript needed). LaTeX/PDF compile checks are skipped, not failed, when no TeX is on `PATH`. To exercise it on a conversation of your own:

```
CLAUDE_PROJECTS=~/.claude/projects SAMPLE_SESSION=<id> python tests/test_archiver.py
```

The suite also checks that this manual and `AGENTS.md` document every CLI flag and that the README's stated check count is current.

9. Building this manual

```
python docs/build_manual.py
```

Renders `USER_MANUAL.md` to `USER_MANUAL.html` and `USER_MANUAL.pdf` with pandoc (and xelatex for the PDF) when available, otherwise with the archiver's own Markdown renderer for the HTML and a note that the PDF was skipped. The built files are committed so readers need no tooling.